



Summer of Bitcoin 2026

Setting up distributed backend storage over yggdrasil mesh network

Project Proposal By

Harsh Chandwani

Email Address	harsh9425181074@gmail.com
Github	https://github.com/harsh04044
Linkedin	https://www.linkedin.com/in/harsh-chandwani-b9a48238a/
Phone	+91 9425181074
TimeZone	Indian Standard Time (IST) (GMT + 5:30)
Discord	harsh040444 (username)
University	Indian Institute of Technology Madras
Idea Link	IDEA LINK FOR THE PROPOSAL

INDEX

1. Overview and Rationale 3

2. Goals and Expected Outcomes 5

3. Architecture 7

4. Technical Implementation Approach 11

5. Implementation Timeline 14

6. Testing Strategy 17

7. Documentation Plan 18

8. Challenges and Mitigations 18

9. Capacity Planning and Durability 19

10. Pre-Proposal Contributions 20

11. About Me 21

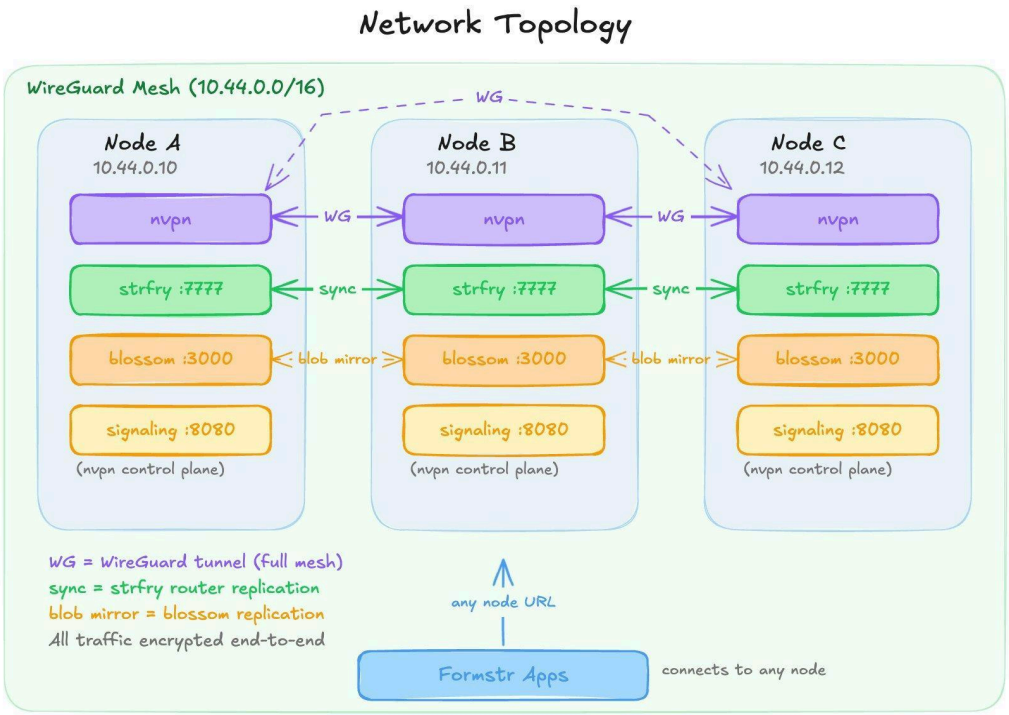
12. Availability and Commitment 21

13. References 21

1. Overview and Rationale

Formstr is a suite of Nostr-based apps: forms, polls, documents, file storage, and calendar. Four of these need file storage, which currently relies on public Blossom servers like blossom.primal.net. Form responses can contain sensitive data (resumes, contracts, identification documents), and all of it goes through centralized third-party servers with no redundancy.

This project replaces that with a private, distributed storage backend on nostr-vpn. The idea is to run a Nostr relay (strfry) and Blossom server on each node of a WireGuard mesh, with events and blobs replicating between them based on configurable policies. The Formstr apps barely need to change since they already talk to Blossom over HTTP with kind 24242 auth and encrypt files with NIP-44 before uploading. Switching to mesh storage comes down to changing a URL in about 5 files. The relay and Blossom server URLs are the entry points directly, there's no gateway sitting in between.



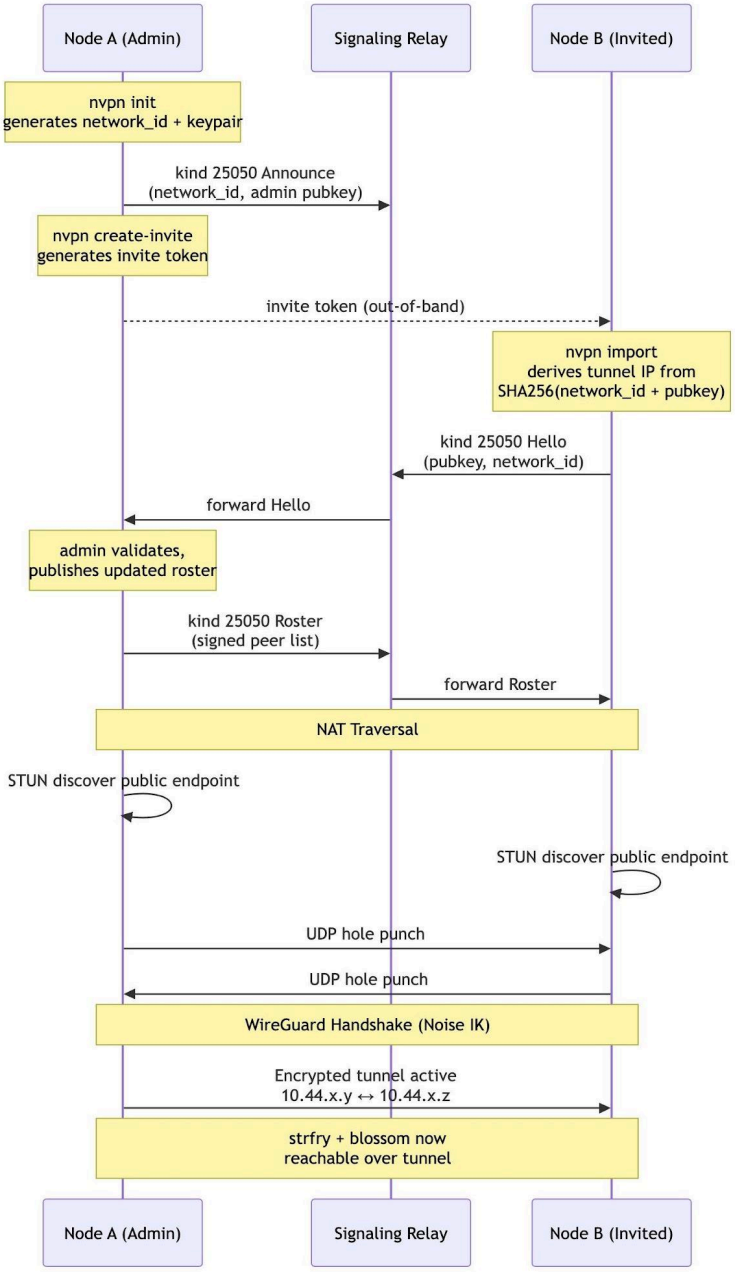
Why nostr-vpn

I looked at three mesh options (Yggdrasil, FIPS, nostr-vpn) and shared a detailed comparison with Abhay. We decided on nostr-vpn together after going through the tradeoffs:

- **Nostr-native signaling:** peer discovery runs over standard Nostr relays using kind 25050 events. No static peer lists, unlike Yggdrasil and FIPS which need IP:port in config files.
- **Private by default:** the mesh is invite-only with admin/participant roles. Nodes are added via nvpn add-participant --publish. Yggdrasil is a public mesh where you'd set up privacy yourself.

- **WireGuard data plane:** encrypted tunnels via boringtun (a userspace WireGuard implementation). NAT traversal is built in with three fallback layers: STUN for public endpoint discovery, UDP hole punching, and relay operator fallback.
- **Version pinning:** on a private mesh we lock to a working version. No need to track upstream changes since all mesh participants are our own nodes.

The diagram below shows what happens when a new node joins the mesh: the admin initializes and creates an invite, the new node imports it, they exchange kind 25050 signals through a signaling relay, work out NAT traversal, and establish a WireGuard tunnel.



I also looked at the alternatives. FIPS has the right vision with npub-native addressing, but its relay discovery isn't actually built yet. I went through the source and every transport's discover() method returns empty. Yggdrasil is more mature but its ed25519 identity system has zero overlap with nostr's secp256k1, which would mean managing a completely separate key layer.

2. Goals and Expected Outcomes

A. Distributed Storage Mesh

Set up a multi-node nostr-vpn mesh where each node runs strfry (nostr relay) and a Blossom server.

- Each node runs four Docker services:
- The nvpn daemon handles the WireGuard tunnel and peer discovery over nostr relays
- strfry stores events (form templates, responses, metadata)
- blossom handles blob storage with kind 24242 auth
- A lightweight signaling relay handles the nostr-vpn control plane
- strfry and blossom share the nvpn container's network namespace via Docker's network_mode, so they automatically bind to the tunnel IP

outcome: form responses and uploaded files are stored across multiple nodes. If one node goes down, others keep serving.

B. Selective Replication

Replace full relay mirroring with policy-based sync using strfry router. Not every node should get every event.

- kind 30168 (form templates): syncs to all nodes, needed everywhere for rendering
- kind 1069 (form responses): syncs only to designated storage nodes, high volume and sensitive
- kind 36363 (blossom server lists): syncs to all nodes for service discovery
- kind 24242 (auth events): not synced at all, simply omitted from the router config

For catching up after downtime, negentropy sync runs on startup or via cron. It figures out what's different between two relays using set reconciliation and only transfers what's missing, not the whole event set.

Replication policies can also be usage-based. Highly requested forms can have their responses replicated to the gateway node for faster access, while low-traffic forms stay on storage nodes only. strfry router's plugin system supports this with a filter script that checks request frequency and decides whether to replicate.

Nodes can also be designated by capability. High-bandwidth nodes with fast storage handle active traffic, while lower-spec nodes serve as redundancy-only replicas. The strfry router config already supports this since each stream specifies which node URLs to replicate to.

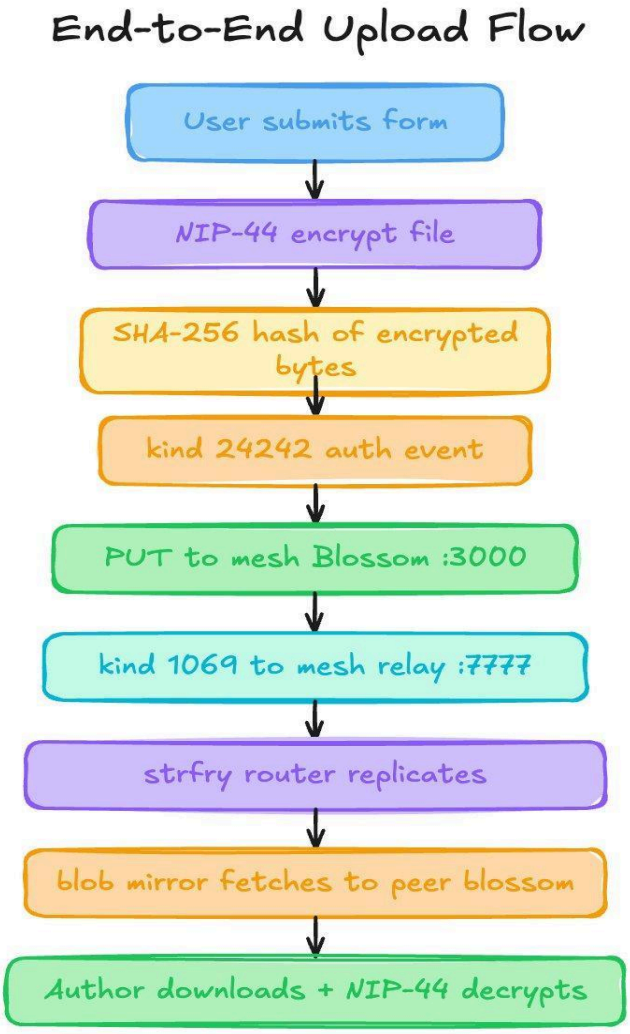
C. Formstr App Integration

The goal here is to connect all four Formstr apps to mesh storage. Every app already talks to Blossom over HTTP and signs uploads with kind 24242 auth events. The integration is really just a URL swap. Each app's encryption model stays completely untouched:

- **nostr-forms:** encrypts files so only the form author can read them (NIP-44 v2, using the responder's key and the form author's key).
- **nostr-polls:** no encryption needed since poll images and videos are public content.
- **nostr-docs:** encrypts each file with a random key. The key is stored inside the document itself, not on any server.
- **formstr-drive:** generates a random key for each file and encrypts it to itself. The decryption key is saved in the file's metadata.

All apps will import from @formstr/storage, which is a shared package that handles the Blossom HTTP layer and auth. Each app does its own encryption before calling upload.

End-to-end data path for a form file upload:



D. Service Discovery and Failover

Two of the Formstr apps (nostr-forms and formstr-drive) already look for Blossom servers by querying kind 36363 events from relays. So all we need to do is publish a 36363 event on the mesh relay that lists the mesh Blossom URLs. The apps pick them up through their existing discovery code. When we add or remove a node from the mesh, no app config changes are needed.

None of the Formstr apps have failover today. If a Blossom server is down, the upload just fails. This project adds the first failover mechanism: the BlossomClient tries the mesh server first with a configurable timeout, and if it fails or times out, falls back to public servers like nostr.download and blossom.oxtr.dev.

3. Architecture

Per-Node Stack

Each mesh node runs as a Docker Compose stack:

- The nvpn container creates a WireGuard tunnel. Each node gets a unique tunnel IP derived from the network ID and its public key
- strfry and blossom share the nvpn container's network namespace using Docker's network_mode: "service:nvpn", so they automatically bind to the tunnel IP
- The signaling relay is a lightweight in-memory nostr relay from the nostr-vpn repo, used only for the control plane

Docker Compose config from the working PoC:

```yaml
# docker-compose.yml
services:
  relay:
    build: ./nostr-vpn
    command: ["nostr-vpn-relay", "--bind", "0.0.0.0:8080"]
    networks:
      mesh: { ipv4_address: 10.203.0.2 }

  node-a:
    build: ./nostr-vpn
    cap_add: [NET_ADMIN]
    devices: [/dev/net/tun:/dev/net/tun]
    depends_on: [relay]
    networks:
      mesh: { ipv4_address: 10.203.0.10 }

  strfry-a:
    image: ghcr.io/hoytech/strfry:latest
    network_mode: "service:node-a"
    volumes:
      - ./strfry.conf:/etc/strfry.conf:ro
      - strfry-data-a:/app/strfry-db

  blossom-a:
    image: ghcr.io/hzrd149/blossom-server:latest
    network_mode: "service:node-a"
    volumes:
      - ./blossom-config.yml:/app/config.yml:ro

networks:
  mesh:
    driver: bridge
    ipam:
      config:
        - subnet: 10.203.0.0/24
```

Network Layout

Two network layers:

- Docker bridge: inter-container communication within a host
- VPN tunnel: mesh traffic between nodes. Each node gets a unique mesh IP, derived from the network ID and the node's public key
- MagicDNS is built into nostr-vpn, so each peer gets a human-readable name (e.g., storage-us.nvpn) that resolves to its tunnel IP via a local DNS server

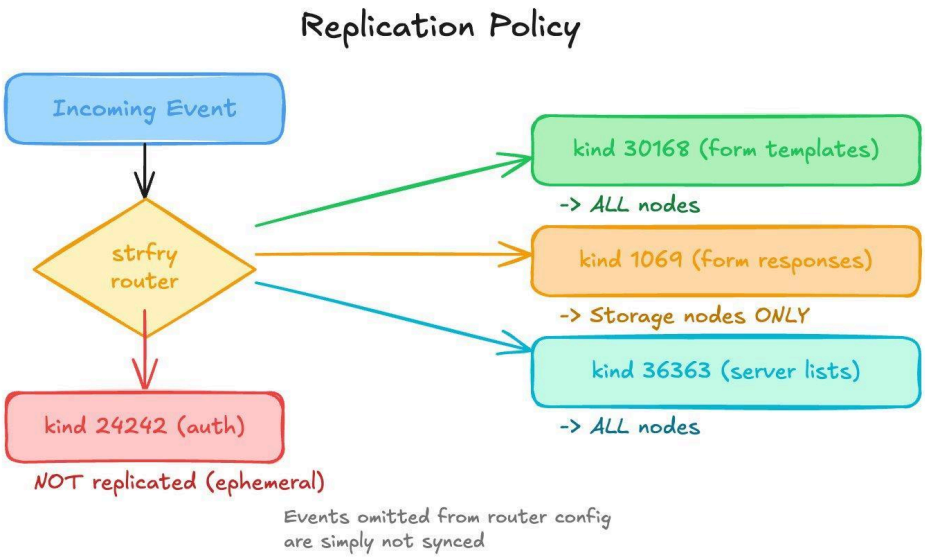
For app access, the relay and Blossom URLs are the entry points, there's no separate gateway in between. The native apps (polls, docs, drive) are Capacitor APKs and could run nostr-vpn directly for mesh access. The web app (formstr.app) connects to publicly reachable relay/blossom URLs on the mesh nodes.

Replication

In the PoC I used strfry stream for quick validation. For production, the plan is to use strfry router instead, which is the officially supported replacement. It adds per-stream filters, plugin support, and hot-reload without needing to restart strfry.

strfry router config for selective replication:

```
# formstr-router.config (production, Phase 2)
streams {
    form-templates {
        dir = "both"
        filter = { "kinds": [30168] }
        urls = ["ws://10.44.0.10:7777",
"ws://10.44.0.11:7777"]
    }
    form-responses {
        dir = "both"
        filter = { "kinds": [1069] }
        urls = ["ws://10.44.0.11:7777"]   # storage nodes only
    }
    server-lists {
        dir = "both"
        filter = { "kinds": [36363] }
        urls = ["ws://10.44.0.10:7777",
"ws://10.44.0.11:7777"]
    }
    # kind 2422 omitted = not replicated
}
```



Blob mirroring runs as a sidecar process alongside strfry:

- Subscribes to the local relay with filter `{kinds: [1069]}`
- For each form response event, extracts the SHA-256 blob reference from the tags
- Checks local blossom (GET /{sha256}, 404 means missing)
- For missing blobs, fetches from peer blossom nodes over the tunnel using a pre-authorized service keypair for kind 24242 auth
- Uploads the fetched blob to local blossom
- Failed fetches retry with exponential backoff

negentropy sync acts as a safety net. It runs on startup and via cron, using set reconciliation to efficiently diff two relays and only transfer what's missing. This catches anything the live router stream might have missed during network hiccups or brief disconnects.

Write Path

When a user submits a form response with a file attachment:

- The app encrypts the file with NIP- 44 and computes the SHA-256 hash of the encrypted bytes
- Creates a kind 24242 auth event and PUTs the encrypted blob to mesh blossom on port 3000
- Publishes a kind 1069 event to the mesh relay on port 7777 with the SHA-256 reference
- strfry router picks up the 1069 event and replicates it to storage nodes
- Blob mirror sidecar on storage nodes sees the new 1069, notices the missing blob, and fetches it from the origin blossom

Read Path

When the form author views a submitted response:

- App queries kind 1069 events from the mesh relay
- Extracts the SHA-256 hash from the event metadata
- Creates a kind 24242 auth event for GET
- Downloads the encrypted blob from mesh blossom
- Decrypts with NIP-44 to recover the original file

Consistency Model

The system is eventually consistent.

- A file uploaded to node A is immediately readable from node A
- It becomes readable from node B after strfry router replicates the kind 1069 event AND blob mirroring fetches the blob, typically under 5 seconds on a healthy mesh
- During this window, reads from node B will fail and trigger failover to the upload-origin node or public Blossom
- Negentropy sync catches any events missed by the live router stream

Security

Here's the threat model: mesh nodes are trusted since they're invite-only and the admin controls the roster. The transport between nodes is untrusted, which is why WireGuard encrypts everything. Storage at rest might be compromised, but that's fine because files are encrypted before they ever leave the app. Even if someone gets access to a blossom server's disk, they can't read anything. The signaling relay is also untrusted, it can see metadata like who's talking to whom and when, but it can't read the actual message content.

- **Application layer:** files are encrypted with NIP-44 v2 before upload, completely independent of the mesh. Even if someone gets access to a blossom server's disk, they only see encrypted data.
- **Mesh transport:** all traffic between nodes goes through WireGuard tunnels. If someone intercepts packets between nodes, they only see encrypted WireGuard frames.
- **Control plane:** the signaling messages that nostr-vpn uses to coordinate peers are encrypted per-recipient, signaling relay forwards them but can't read them.
- **Access control:** the mesh is invite-only. The admin creates an invite link, the new node imports it. The admin maintains a signed roster that all peers verify. To remove a node, the admin publishes an updated roster and all peers automatically disconnect from the removed node.

4. Technical Implementation Approach

Docker, nostr-vpn, strfry, and blossom already exist as standalone tools. What I'm building is the layer that connects them: a shared storage package for the Formstr apps, a blob mirroring system to keep files synced across nodes, failover logic, and the integration to connect each app to the mesh.

formstr/storage Package

This is the main deliverable on the client side. It's a TypeScript npm package that pulls out the Blossom upload/download code that's currently copy-pasted between nostr-forms and formstr-drive. I've already started this work (PR #2 on common-packages).

The package exports:

- **BlossomClient:** handles the HTTP communication with Blossom servers (upload, download, delete). Right now this code is copy-pasted between nostr-forms and formstr-drive with almost identical logic.
- **createAuthEvent**: builds the signed authorization events that Blossom servers require for uploads and downloads. Works with both raw secret keys and browser extension signers (NIP-07).
- **NIP-44 helpers:** `encryptFileToAuthor()` and `decryptFileFromUploader()` for encrypting large files. I verified these byte-for-byte against the nostr-forms reference implementation to make sure they're compatible.

New convenience wrappers for the different encryption models across apps:

- **upload ():** encrypts the file with NIP-44 and uploads it. Used by nostr-forms and formstr-drive.
- **uploadRaw ():** no encryption, direct upload. Used by nostr-polls.
- **uploadSymmetric ():** AES-GCM with random key, then upload. Used by nostr-docs.

Per-App URL Changes

Each app just needs its default Blossom server list updated to include the mesh URL. The changes are small. Here's what each one looks like:

nostr-forms (FileUploadSettings.tsx):

```
export const DEFAULT_SERVERS = [
    "https://nostr.download",
    "https://blossom.oxtr.dev",
+   MESH_BLOSSOM_URL,
  ];
```

nostr-polls (blossomService.ts):

```
- const DEFAULT_BLOSSOM_SERVER = "https://blossom.primal.net";
+ const DEFAULT_BLOSSOM_SERVER = MESH_BLOSSOM_URL;
```

nostr-docs (BlossomContext.tsx):

```
- const DEFAULT_SERVERS = ["https://blossom.primal.net"];
+ const DEFAULT_SERVERS = ["https://blossom.primal.net",
MESH_BLOSSOM_URL];
```

formstr-drive (BlossomServerProvider.tsx):

```
const DEFAULT_SERVERS = [
    "https://blossom.primal.net",
    "https://nostr.download",
    "https://blossom.oxtr.dev",
+   MESH_BLOSSOM_URL,
  ];
```

Both nostr-forms and formstr-drive already discover Blossom servers by querying kind 36363 events from relays. So publishing a 36363 event on the mesh relay is all it takes for mesh servers to show up in the dropdown, no code changes needed.

Failover Logic

This is the only piece of genuinely new client-side logic in the project. It gets added to BlossomClient. The idea is straightforward: try the mesh server first with a 5-second timeout, and if it fails, fall back to public Blossom servers. nostr-docs already has a parallel multi-server pattern using Promise.allSettled that I can use as a reference.

```
async function uploadWithFailover(file: Uint8Array,
authHeader: string) {
  try {
    return await blossomClient.upload(file, {
      server: MESH_BLOSSOM_URL, timeout: 5000
    });
  } catch {
    return await blossomClient.upload(file, {
      server: DEFAULT_PUBLIC_SERVER
    });
  }
}
```

Blob Mirroring Sidecar

This is the biggest chunk of new code in the project, probably 200-400 lines. It's a Node.js or shell script that runs alongside strfry on each storage node. Here's what it does:

- Subscribe to the local strfry relay via WebSocket with filter `{kinds: [1069]}`
- For each incoming form response event, extract the SHA-256 blob hash from the response tags
- Check local blossom: GET /{sha256}. If 404, the blob is missing locally
- For missing blobs, iterate through peer blossom nodes (known from mesh config), create a kind 24242 GET auth event signed with a pre-authorized service keypair, and download the blob
- Upload the downloaded blob to local blossom with a kind 24242 PUT auth
- On failure, retry with exponential backoff (1s, 2s, 4s, 8s, max 60s)
- Log all operations for monitoring. Track success/failure counts.

Migration Script

A one-time script to move existing data from public Blossom servers to the mesh:

- Walks through kind 1069 events on the relay
- Extracts the SHA-256 hash and source server from each event's metadata
- Downloads the blob from the public Blossom server
- Re-uploads to the mesh Blossom with fresh kind 24242 auth
- Runs in batches and is idempotent (skips blobs that already exist on the mesh)

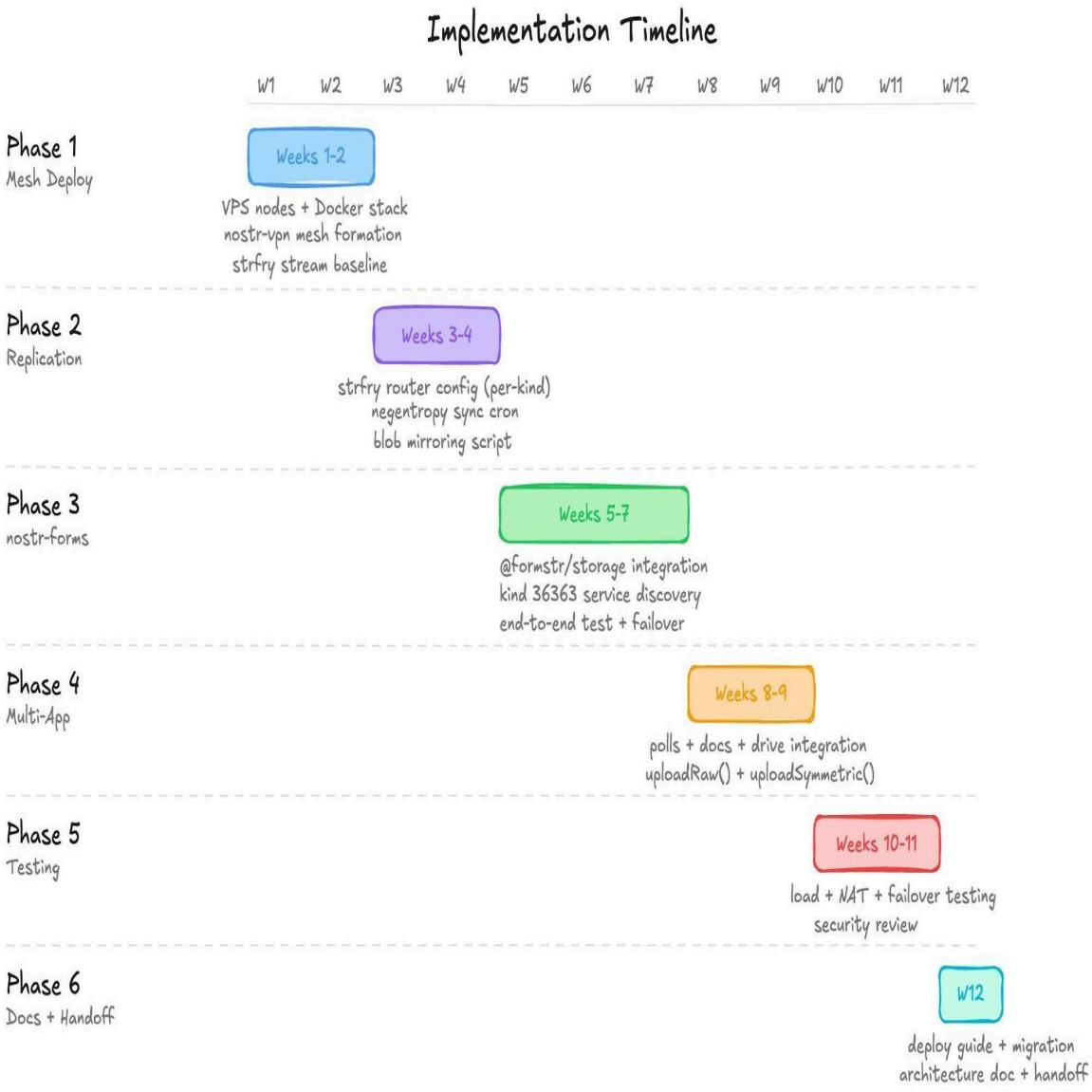
Monitoring and Health Checks

A set of scripts that run via cron to monitor mesh health:

- nvpn status --json: check peer connectivity, tunnel status, last handshake time
- strfry relay info: event counts per kind, database size, connection count
- Blossom health: HTTP GET to each node's blossom, check response time
- Replication lag: publish a test event on one node, measure time until it appears on peers
- Disk usage alerts: warn when blossom storage exceeds 80% of available disk

5. Implementation Timeline

My academic schedule is flexible, so I can commit 45 hours/week consistently, with room to push to 50 if a deadline is tight. I'm in IST (UTC+5:30). I'll post weekly progress updates on signal with mentors and flag blockers as they come up.



Week 1: VPS + Mesh Formation

Spin up 2 VPS nodes, install Docker, and get the nostr-vpn mesh running between them. By end of week, both nodes should be able to ping each other over the tunnel.

- **Verify:** both peers show as online in nvpn status, tunnel ping works. Push the deployment config to repo.

Week 2: Services + Baseline Replication

Add strfry and blossom to each node, sharing the tunnel network. Set up baseline replication between the two relays.

- **Verify:** both relays and both blossom servers are reachable over the tunnel. Publish a test event on one node, confirm it shows up on the other.
- **Repo artifact:** deployment scripts and config files.

Week 3: strfry Router Migration

Migrate from strfry stream to strfry router with per-kind filtering. Configure which event types go to which nodes.

- **Verify:** form templates replicate to all nodes, form responses only to storage nodes, auth events don't replicate at all.
- **Repo artifact:** router config with documented filter rules.

Week 4: Negentropy + Blob Mirroring

Set up negentropy sync as a cron job for catch-up after downtime. Write and deploy the blob mirroring sidecar.

- **Verify:** take a node offline, publish events to the other, bring it back, and confirm negentropy catches everything up. Upload a file to one node's blossom and confirm the mirror copies it to the other.
- **Repo artifact:** negentropy cron config, blob mirroring script, sync policy docs.

Week 5: formstr/storage + Service Discovery

Finalize the formstr/storage package with all three upload wrappers. Point it at the mesh endpoint and publish a kind 36363 event so apps can discover the mesh servers.

- **Verify:** nostr-forms picks up the mesh server in its dropdown automatically. A test upload to mesh blossom succeeds.

Week 6: nostr-forms End-to-End

Full integration test with nostr-forms against mesh storage.

- **Verify:** create a form with a file upload field, submit a response with a PDF, and trace it all the way through: encrypted upload to mesh blossom, event published and replicated, blob mirrored to peer node and the form author can download and decrypt it from the other end.

Week 7: Failover Logic

Implement failover in BlossomClient: try mesh first, fall back to public servers if it's down.

- **Verify:** block mesh access, confirm uploads still work via public fallback. Restore mesh, confirm it switches back.
- **Repo artifact:** updated formstr/storage with failover logic.

Week 8: nostr-polls + nostr-docs

Add mesh URL to nostr-polls and nostr-docs.

- **Verify polls:** create a poll with an image, confirm it uploads to mesh and loads correctly.
- **Verify docs:** insert a file in a document, confirm it uploads to both mesh and public servers.
- **Repo artifact:** `uploadRaw()` wrapper added to formstr/storage.

Week 9: formstr-drive + Wrappers

Add mesh URL to formstr-drive. Add the `uploadSymmetric()` wrapper to formstr/storage.

- **Verify:** upload a file in formstr-drive, confirm it lands on mesh blossom encrypted. Download from a different node, confirm it decrypts.
- **Verify:** formstr-drive picks up mesh servers automatically via kind 36363 discovery.
- **Repo artifact:** all 4 apps committed with mesh URLs, all wrappers in formstr/storage.

Week 10: Load + NAT Testing

- **Load test:** simulate 20 concurrent uploads from 5 clients. Measure upload latency p50/p95, relay event throughput, blob download speed across nodes.
- **NAT test:** try forming the mesh across different network environments (VPS, home NAT, mobile hotspot). Document what works and what doesn't.

Week 11: Failover + Security Testing

- **Failover test:** kill a node, confirm the others keep serving. Bring it back, measure how fast it catches up.
- **Replication test:** publish different event kinds, verify the router policies are working correctly.
- **Security review:** check that no keys leak in logs, files on disk are encrypted, and no plaintext traffic leaves the tunnel.
- **Repo artifact:** test report with numbers, hardened config.

Week 12: Documentation + Handoff

- **Deployment guide:** how to set up a new mesh node from scratch.

- **Migration script:** move existing files from public Blossom servers to the mesh.
- **Architecture doc:** full stack explanation with diagrams.
- **Handoff:** update all repo READMEs with mesh storage config.

6. Testing Strategy

Testing runs throughout the project, not just in the dedicated testing weeks. Every week's deliverable includes a concrete verification step. The testing weeks (10-11) focus on cross-cutting concerns that only make sense once the full system is integrated.

Unit Tests

- formstr/storage: verify encryption roundtrips work, auth events are structured correctly, and the HTTP client handles uploads and downloads properly
- Blob mirroring: verify it correctly identifies missing blobs, fetches them from peers, and retries with proper backoff on failure
- Failover: timeout behavior, fallback server selection

Integration Tests

- End-to-end upload: take a file through the full cycle from encryption to upload to replication to download and decryption
- Replication correctness: publish different event kinds and verify that each one only shows up on the nodes it's supposed to
- Failover: block mesh, verify public fallback, restore mesh, verify mesh resumes

Load Tests

- 20 concurrent uploads from 5 simulated clients
- Measure upload latency, how many events the relay can handle per second, and download speed across nodes

Infrastructure Tests

- NAT traversal across home NAT, mobile hotspot, VPS-to-VPS
- Node failure and recovery: kill a node, measure negentropy sync time
- Disk failure: delete a node's data directory, verify full recovery from peers

7. Documentation Plan

Deployment Guide

Step-by-step instructions for provisioning a new mesh node: VPS setup, Docker installation, nvpn init, invite creation and import, Docker Compose configuration, strfry router setup, blossom configuration.

Architecture Doc

Full stack explanation with diagrams: nostr-vpn mesh layer, strfry replication layer, blossom storage layer, @formstr/storage client layer. Written for someone who needs to maintain or extend the system.

Node Operations

How to add/remove nodes from a running mesh (nvpn add-participant --publish), monitor health (nvpn status --json, nvpn netcheck) and recover from downtime (negentropy sync).

Migration Guide

How to move existing Formstr data from public Blossom servers to the mesh. Includes the migration script with usage instructions.

formstr/storage API Docs

Package README with usage examples for `upload()`, `uploadRaw()`, `uploadSymmetric()`, and the failover configuration.

8. Challenges and Mitigations

Challenge	Mitigation
nostr-vpn is a young project (v0.3.11)	Since it's a private mesh, we pin to a version that works and only update if there's a security fix.
strfry stream is deprecated	The PoC uses stream to prove replication works quickly. In Week 3, we migrate to strfry router which is the supported replacement and does everything stream does plus filtering.
NAT traversal can fail	nostr-vpn has three fallback layers: STUN, hole punching, and relay operator. For the production deployment on VPS, NAT isn't even an issue since direct UDP works.
Tunnel IP collisions	The address space is large enough that collisions are negligible for meshes under 50 nodes. For bigger deployments, manual IP assignment is an option.
nostr-vpn has a single maintainer	The project is MIT licensed. We pin to a working version on our private mesh, so even if development stalls, we have a frozen

	working binary. The mesh transport layer is designed to be swappable if needed.
No failover in Formstr apps today	This project adds the first failover mechanism. If the mesh is down, uploads fall back to public Blossom servers automatically.
What if a node's disk dies	Every event and file exists on at least 2 nodes. If a disk is lost, negentropy rebuilds the relay and the blob mirror re-fetches files from surviving nodes.

9. Capacity Planning and Durability

Capacity

For the initial deployment, each VPS would be a Hetzner CX22: 2 vCPU, 4GB RAM, 80GB SSD, around 4 EUR/month. strfry uses about 50MB of RAM when idle and scales with event count. Blossom storage grows with blob volume, 80GB can hold roughly 8,000 files at 10MB average. When a node's storage fills up, an LRU eviction policy removes the least recently accessed blobs first. Evicted blobs stay on peer nodes and can be re-fetched on demand if needed. For Formstr's use case (form responses with occasional file attachments), this should be more than enough for the first year. To scale beyond that, add more nodes to the mesh and adjust replication policies to shard responses by form-creator pubkey.

Durability

- **Events:** every event lives on at least 2 nodes. If a node goes offline for days, it catches up in seconds on restart since negentropy only transfers what's missing.
- **Blobs:** every file exists on at least 2 nodes. If a node's disk is lost, the blob mirror re-fetches everything from the surviving nodes.
- **Recovery target:** if a node goes down and comes back after missing 100 events and 10 file uploads, it should catch up on events within 30 seconds and recover all blobs within 2 minutes.
- **Not covered:** geographic redundancy. All nodes in the same Hetzner region for initial deployment. Cross-region replication is a future enhancement.

10. Pre-Proposal Contributions

PR	Repo	Description	Status
#2	common-packages	Extracted Blossom upload/download into formstr/storage package. BlossomClient, kind 24242 auth, NIP-44 encryption helpers.	Open
#186	nostr-polls	Added https intent filter for pollerama.fun so shared poll links open in the app. Tested on Pixel emulator. Released in Pollerama 1.8.2. (Deep Linking)	**Merged**
#164	nostr-polls	Fixed the broken Jest test pipeline. Added polyfills for nostr-tools and MUI, replaced the default CRA test with a proper smoke test that renders the Pollerama app.	**Merged**
#165	nostr-polls	Added tests for poll event creation, vote tallying, and component rendering. Builds on the test pipeline fix from PR #164.	Open
#12	nostr-docs	Added the full **add-comment** flow for nostr-docs, with collapsible sidebar.	**Merged**
#26	nostr-docs	Added **find and replace** functionality to the document editor.	Open
#18	nostr-docs	Set up vitest for nostr-docs and added the first utility tests.	Open

nostr-vpn Distributed DB PoC

I built a Docker-based PoC with 2 nostr-vpn nodes, each running strfry and blossom. The mesh connects with 0.2-0.8ms ping between tunnel IPs, event replication works in both directions, and both Blossom servers respond over the tunnel.

Repo: https://github.com/harsh04044/nostr-vpn-poc
Walkthrough: https://youtu.be/vXMyklkoTEo

Mesh Comparison

I evaluated Yggdrasil, FIPS, and nostr-vpn for Formstr's use case. I went through all three codebases: FIPS relay discovery turned out to be unimplemented, Yggdrasil is actively maintained. Shared the comparison doc with Abhay, which led to the nostr-vpn decision.

11. About Me

I'm Harsh Chandwani, a 2nd year BS student at Indian Institute of Technology Madras. I started with full-stack web development about a year and a half ago, then moved toward backend and systems work. I work mostly with TypeScript, React, Docker, and I've been picking up Rust.

I have **70+ merged open-source contributions** across different projects.

For Formstr specifically, I have 7 PRs across three repos (nostr-polls, nostr-docs, common-packages), including a deep linking, add a comment flow, find & replace features. Outside of PRs, I built the nostr-vpn distributed DB PoC, extracted the formstr/storage package.

Before writing this proposal I spent time reading each Formstr app's upload/download code, tracing NIP-44 encryption flows, understanding kind 24242 auth. The specifics in this proposal come from that codebase work and from discussions with Abhay on Signal.

A full list of my open-source contributions can be found here.

12. Availability and Commitment

My college has a 0% attendance requirement, so I can work full time on this regardless of whether the semester is running. During the coding period I can put in 45-50 hours per week consistently. I am in IST (UTC +5:30) and can adjust my schedule for calls or reviews with mentors in other timezones.

I have one exam on May 10. Other than that, I have no academic commitments or travel planned during the coding period. If anything else comes up I'll flag it early.

I am 100% dedicated to formstr and have nothing else planned this summer

13. References

- nostr-vpn: https://github.com/mmalmi/nostr-vpn
- strfry: https://github.com/hoytech/strfry
- Blossom (BUD spec): https://github.com/hzrd149/blossom
- Formstr: https://github.com/formstr-hq
- NIP-44: https://github.com/nostr-protocol/nips/blob/master/44.md
- nostr-vpn PoC: https://github.com/harsh04044/nostr-vpn-poc
- @formstr/storage PR: https://github.com/formstr-hq/common-packages/pull/2
- Mesh comparison doc: https://docs.google.com/document/d/1hHJwBgMcdYE8xE9TZS0l11rpy2V82F-Wamn1JbkpGcs/edit

14. Post SoB

After Summer of Bitcoin ends I plan to stay involved with Formstr in a few ways.

The blob mirroring sidecar and failover logic will likely need tuning once real users start uploading files through the mesh. I'll keep an eye on issues related to these and fix bugs as they come up.

The formstr/storage package is meant to be the shared foundation for all Formstr apps. As new apps get added or existing ones change their upload patterns, the package will need updates. I plan to maintain it and add new wrappers as needed.

I'd also like to help with the strfry router migration to production and contribute to other parts of the Formstr ecosystem beyond storage.